

 Département des Technologies de l'Informatique	
EXAMEN	
Unité d'Enseignement : <i>Conception Orientée Objet</i>	Parcours : Développement des Systèmes d'Information
Elément Constitutif (ECUE) : <i>Modélisation Objet UML</i>	Classes : DSI 21, 22, 23
Date : 08 janvier 2024	Nombre de pages : 3
Durée : 1h30min	Documents Autorisés : Néant
Proposé par : <i>Mme Hela GABSI</i>	

NB :

- Le détail des réponses à tous les exercices doit figurer dans la feuille de réponse.
- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

Exercice 1 : Étude de cas (16pts)

On considère un parc de stationnement payant. Le stationnement est payant à l'heure, le tarif est de 2 dinars /heure.

L'entrée et la sortie du parking sont contrôlées par des barrières.

Lorsqu'un usager arrive au parking, une machine qui fait partie du système de paiement lui délivre un ticket sur lequel est inscrit un numéro unique. La barrière d'entrée le laisse alors entrer, puis il gare son véhicule.

Lorsque l'usager en question revient pour récupérer son véhicule, il passe d'abord par une caisse automatique qui fait également partie du système de paiement. Il introduit son ticket dans la caisse, le ticket est analysé de façon automatique (lu par un scanner), puis la caisse automatique indique la somme que l'usager doit pour son stationnement.

Ce dernier paie cette somme en liquide ; la machine lui rend la monnaie, puis lui délivre un ticket de sortie lui-même numéroté, valable 15 minutes. Il récupère alors son véhicule et se rend à la sortie du parking. Il introduit alors son ticket de sortie dans une machine qui fait également partie du système de paiement. Après analyse du ticket de sortie, la barrière se lève, et l'usager sort du parking.

Certains usagers utilisent le parking de façon quotidienne et sont abonnés au mois. Il bénéficie d'un mode de paiement spécial : l'abonnement.

Un abonné paie une somme forfaitaire de 50 dinars tous les mois, puis seulement 0,5 dinars de l'heure. Chaque abonné bénéficie à ce titre d'une carte d'accès permanent qui l'identifie, et lui permet d'ouvrir les barrières d'entrée et de sortie du parking. L'abonné ne passe jamais par la caisse automatique. Le compte de ses heures de stationnement est fait automatiquement par le système qui enregistre ses heures d'arrivée et de départ. Au 1^{er} de chaque mois, sa facture lui est envoyée automatiquement par le système de paiement.

Par ailleurs, lorsqu'un usager perd son ticket de parking, il doit s'identifier à l'opérateur et lui payer la somme de 20 dinars ; l'opérateur lui ouvre alors la barrière de sortie.

DS

Unité d'Enseignement : Programmation Objet Avancée	Classes : DSL, ZX
Durée : 1 Heure	Nombre de pages : 4
Proposé par : Mme Yosra KASSIS, M. Mossaab BOUKHCHIM.	Documents autorisés : NON

NB :

- Le détail des réponses de tous les exercices doit figurer dans la feuille d'examen.
- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

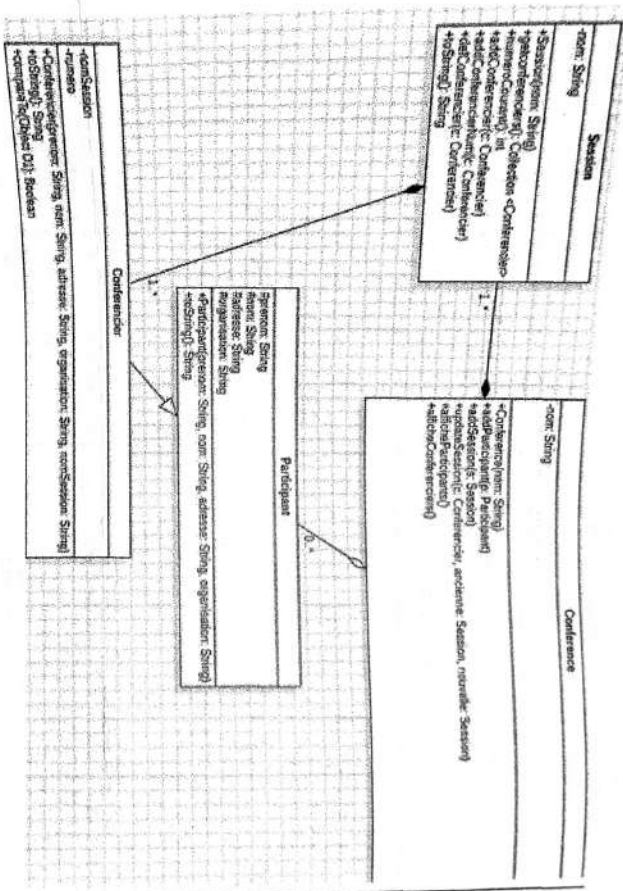
Exercice 1 :

Donner le diagramme de classe correspondant au code JAVA suivant :

<pre>public class Personne { private int cin; private String nom; private String prenom; // Constructeur // Getters & setters }</pre>	<pre>public class Pilote extends Personne { // Constructeur // Getters & setters }</pre>
<pre>public class Vol { private Date dateVol; private Ville villeDepart; private Ville villeArrivee; private ArrayList<Reservation> listeReservations = new ArrayList<Reservation>(); // Constructeur // Getters & setters }</pre>	<pre>public class Ville { private int codeVille; private String nomVille; // Constructeur // Getters & setters }</pre>
<pre>public class Reservation { private int numReservation; private Date dateReservation; private Passenger passager; private Vol vol; // Constructeur // Getters & setters }</pre>	<pre>public class Passenger extends Personne { private int numPassager; private String siege; private ArrayList<Reservation> listeReservations = new ArrayList<Reservation>(); // Constructeur // Getters & setters }</pre>
<pre>public class Avion { private String modele; private int numSerie; private Roue [] tabRoues; public Avion() { this.Roues=new Roue[8]; } //Getters & setters }</pre>	<pre>public class Roue { private int numRoue; // Constructeur // Getters & setters }</pre>

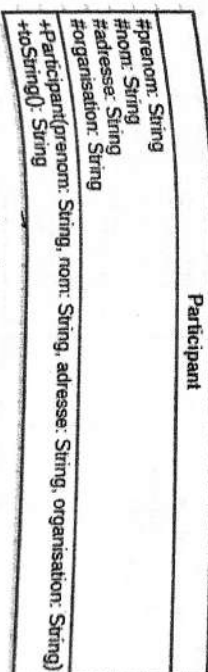
Exercice 2 :

On désire assurer la gestion des participants à une conférence.
Les participants à cette conférence peuvent simplement assister aux diverses sessions, mais aussi peuvent être des conférenciers. Le diagramme de classes ci-dessous décrit brièvement notre future application.



Questions

1. Développer en java la classe Participant



Un participant est représenté par un nom, un prénom, le nom de son organisation, et son adresse.

2. Implémenter en java la classe Conférencier

Conférencier	
-nomSession: String	
-numero: int	
+Conférencier(prenom: String, nom: String, adresse: String, organisation: String, nomSession: String)	
+toString(): String	
+compareTo(Object O1): Boolean	
+getNumero(): int	
+setNumero(): void	

Un conférencier (Hérité de la classe Participant) est représenté par un nom, un prénom, le nom de son organisation, et son adresse, le nom de la session et éventuellement le numéro de passage de la session

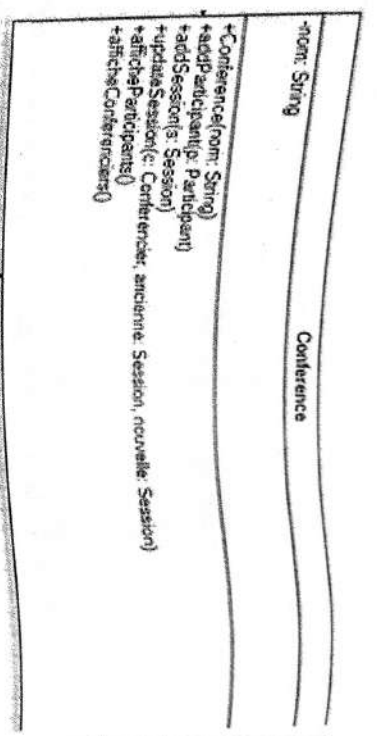
3. Codifier en java la classe Session

Session	
-nom: String	
+Session(nom: String)	
+getConférenciers(): Collection <Conférencier>	
+numeroCourant(): int	
+addConférencier(c: Conférencier)	
+addConférencierNum(c: Conférencier)	
+delConférencier(c: Conférencier)	
+toString(): String	

Les sessions sont représentées par leur nom sous forme d'une chaîne de caractères. Pour chaque session, l'ordre de passage des conférenciers est mémorisé.

Noter bien que :

- La méthode public int numeroCourant () Calcule le numéro de passage le plus élevé de la session.
 - La méthode public void addConférencierNum (Conférencier c) ajoute le conférencier c et lui attribue le numéro de passage le plus élevé.
 - La méthode public void addConférencier (Conférencier c) ajoute le conférencier avec un numéro de passage déjà attribué.
 - La méthode public void delConférencier (Conférencier c) Supprime le conférencier.
4. Développer en java la classe Conférence



Classe principale qui permet la gestion d'une conférence.

Noter bien que :

-La méthode public void updateSession (Conferencier c, Session ancienne, Session nouvelle) Modifie la session d'un conferencier.

Bon Travail 😊



Département des Technologies de l'Informatique

Examen

Unité d'Enseignement : POO	Classes : DS1/2
Durée : 1H30	Nombre de pages : 03
Date : 04/01/2024	Heure de début : 11H
Proposé par : T. DHIFALLAH- M. BOUKHCHIM- H. GHAZEL	Documents Autorisés : Non

N.B : On tiendra compte de la présentation et de la lisibilité des réponses.

Exercice

Compléter les classes nécessaires à l'exécution de ce programme tout en sachant que les classes Directeur, Ingénieur et Technicien dérivent de la classe Employé.

Résultat du programme :

si n = 1 : on obtient à l'écran : je suis l'employé Med et j'ai 29 ans.
 si n = 2 : on obtient à l'écran : je suis le technicien ali, j'ai 35 ans et je suis un cadre moyen.
 si n = 3 : on obtient à l'écran : je suis l'ingénieur Meriam, j'ai 37 ans, je suis un cadre principale et mon salaire est de 10000.
 si n = 5 : on obtient à l'écran : je suis le Directeur Salah, j'ai 49 ans, je suis un haut cadre et mon salaire est de

```
public class TestEmploye{
    public static void main (String[]args)
    {
        int n = 3 ;
        Employe p ;
        switch (n)
        {
            case 1 : p = new Employe ("Med", 29); break;
            case 2 : p = new Technicien ("ali", 35, "cadre moyen");break;
            case 3 : p = new Ingenieur ("Meriam", 37, "cadre principale", 10000);break;
            case 4 : p = new Directeur ("salah", 49, "haut cadre", 30000); break;
        }
        p.affiche ();
    }
}
```

Problème

Dans le cadre de l'informatisation d'une mairie, on veut automatiser le calcul des taxes des locaux de la zone industrielle.
 On distingue deux catégories des locaux : les usines et les locaux administratifs.
 La taxe se calcule différemment selon le type des locaux. Pour cela, on définit les classes : **Usine** et **Administration** et les caractéristiques communes à ces deux classes sont regroupées dans la classe **Local**.

On a donc un schéma de classes où les classes **Usine** et **Administration** héritent de la classe **Local**.

L'objet de ce problème est d'implémenter ce schéma d'héritage et de mettre en œuvre le mécanisme de liaison dynamique.

1) Définir la classe **Local** contenant :

❖ Les attributs suivants :

- propriétaire du type chaîne de caractères et qui correspond au nom du propriétaire,
- adresse du type chaîne de caractères et qui correspond à l'adresse du local,
- surface du type double et qui correspond à la surface du local.

❖ Un constructeur paramétré permettant d'initialiser une instance de la classe **Local**.

❖ Les méthodes :

- **GetType()** : qui renvoie le type de l'objet courant.
- double **Taxe()** : qui permet de calculer le montant de la taxe que doit payer le propriétaire du local à raison de 0,5 dinars par m².
- public String **toString()** : qui permet de retourner le type ainsi que l'état de l'instance, c'est-à-dire les valeurs de ses attributs.
- Void **Affiche()**.

2) Définir la classe **Administration** qui hérite de la classe **Local** et contenant :

❖ Les attributs suivants :

- NbBureau de type entier,
- Restaurant de type booléen.

❖ Un constructeur paramétré permettant d'initialiser une instance de la classe **Administration**.

❖ Redéfinir les méthodes :

- **GetType()** : qui renvoie le type de l'objet courant.
- double **Taxe()** : Le calcul de la taxe d'une **Administration** est différent de celui d'un **Local**, il se calcule en fonction de la surface du local (même tarification que pour un local), du nombre de bureau et de la présence ou non d'un restaurant. On compte supplémentaire 2 dinars/bureau et 50 dinars en cas de présence d'un restaurant.
- public String **toString()**.
- Void **Affiche()**.

3) Définir la classe **Usine** qui hérite de la classe **Local** et contenant :

❖ L'attribut suivant :

- NbEmployés de type entier.

❖ Un constructeur paramétré permettant d'initialiser une instance de la classe **Usine**.

❖ Redéfinir les méthodes :

- **GetType()** : qui renvoie le type de l'objet courant.
- double **Taxe()** : Le calcul de la taxe d'une usine est également différent de celui d'un local. Il se calcule en fonction de la surface occupée par le local (même tarification que pour un local) et du nombre d'employés travaillant dans l'usine. On compte 100 dinars supplémentaires par tranche de 10 employés (on arrondit à la tranche la plus petite).
- public String **toString()**.
- Void **Affiche()**.

Remarque : les constructeurs et les méthodes **Taxe()** et **toString()** des deux classes dérivées devront obligatoirement utiliser les méthodes appropriées de la classe parente.

- 4) Créer une classe **Collection** qui utilise la classe **Local**. Cette classe permet de gérer les locaux d'une mairie. Cette classe est caractérisée par les attributs suivants :
- tab : représente un tableau polymorphe de locaux.
 - Taille : représente le nombre des locaux de la mairie.
 - Max : représente le nombre maximal des locaux de la mairie.
- Ainsi que les méthodes suivantes :
- Un constructeur qui crée une collection ayant un nombre maximum de locaux indiquée en paramètre et initialise la taille de la collection.
 - Void ajouter_Local(Local) qui ajoute un local à la collection.
 - Void Afficher_collection () qui affiche les informations relatives à toutes les locaux de la collection (n'oublier pas d'afficher aussi la valeur de la taxe accordée à chaque local)
 - Collection Liste_Usine () : C'est une méthode qui retourne une collection formée seulement des usines.
 - Collection Liste_Administration () : C'est une méthode qui retourne une collection formée seulement des administrations.
- 5) Créer une classe testcollection contenant la méthode **main ()** dans laquelle vous allez :
- Créer une collection de locaux C1
 - Remplir son tableau par 5 instances des classes Usine et Administration.
 - Pour chaque élément du tableau, afficher ses caractéristiques ainsi que la valeur de sa taxe.
 - Créer une collection C2 et la remplir par les usines de C1
 - L'afficher à l'écran
 - Créer une collection C3 et la remplir par les administrations de C1
 - L'afficher à l'écran

BON TRAVAIL...

المعهد العالي للدراسات التكنولوحيه
بزعوان

المادة : قانون الإعلام
الأستاذة : سراح الملقح
الأفواج : DSI 21/22/23
التاريخ : 24/1/2024

الموضوع :

أشواج الجراسم الألكتروني

حناء مغير



Unité d'Enseignement : Système de Gestion de Base de Données		Classes : DS121, DS122 et DS123
Durée : 1h		Nombre de pages : 4
Date : 03/05/2024		Heure de début : 10H30
Proposé par : Mbarika Ben Youssef et Nidhal Gribaa		Documents Autorisés : NON

Exercice 1 (5 points)

Choisir la (les) réponse(s) correcte(s)

1. Un SGBD :

- ☐ Signifie système de génération de base de données
- ☐ Signifie système de gestion de base de données
- ☐ Assure un accès rapide, efficace et sécurisé aux données
- ☐ Assure la gestion des bases de données réparties
- ☐ Ne peut pas gérer les accès simultanés par plusieurs utilisateurs

2. Une base de données peut être :

- ☐ Logique ☐ Relationnelle ☐ Hiérarchique ☐ Réseau ☐ Entité-Association

3. Pour effectuer des tâches comme la gestion des utilisateurs et des droits d'accès lequel des langages suivants est utilisé ?

- ☐ DCL – Data Control Language ☐ DML – Data Manipulation Language
- ☐ DQL – Data Query Language ☐ DDL – Data Definition Language

4. Une contrainte d'intégrité

- ☐ Est une propriété permettant d'assurer la cohérence des données
- ☐ Une règle qui une fois définie ne peut pas être annulée
- ☐ Permet de définir des contraintes sur la base de données comme la clé primaire, clé étrangère



Unité d'Enseignement : Bases de données		Classes : DS121, DS122 et DS123	
Durée : 1h30		Nombre de pages : 4	
Date : 06/01/2024		Heure de début : 11H	
Proposé par : Mbarika Ben Youssef et Nidhal Gribaa		Documents Autorisés : NON	

Exercice 1 : (4 points)

Une personne peut ouvrir un ou plusieurs comptes dans une agence bancaire.

Elle doit donner son nom, son adresse, et éventuellement son numéro de téléphone.

Chaque compte a une seule date d'ouverture, un numéro d'identification de compte,

et un type (compte chèques, compte d'épargne, etc). Chaque compte appartient à un

seul titulaire. Chaque agence bancaire a un nom, un numéro d'agence, une adresse

et un numéro de téléphone. Une opération est effectuée par une seule personne (le

titulaire du compte ou un tiers) et concerne au plus deux comptes : le compte

émettant et le compte recevant l'opération. Une opération a un numéro

d'identification, une date, un lieu, un type qui indique la nature de l'opération : retrait

par carte bancaire, versement de salaire, encaissement ou paiement de chèque,

virement de compte à compte, etc. Le solde d'un compte est établi à une date

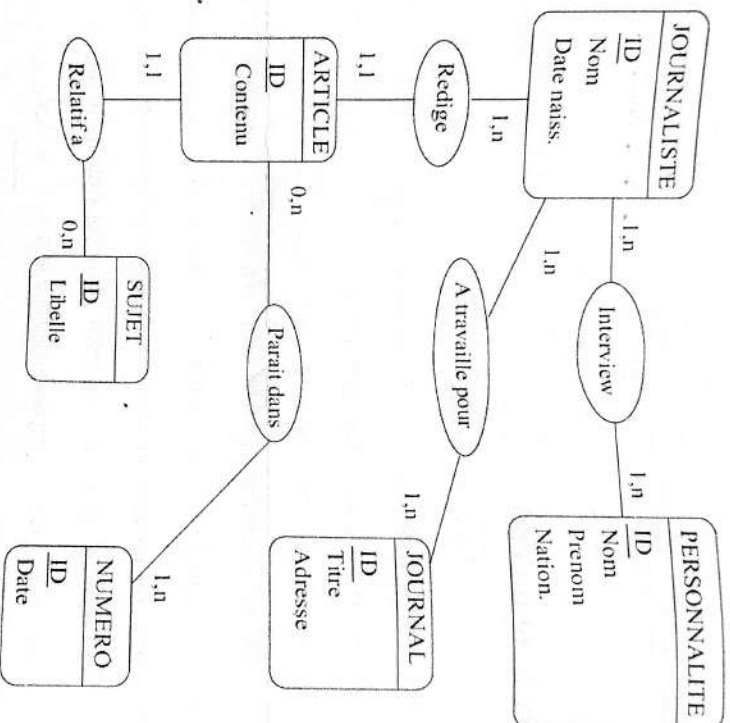
précise, au début de chaque mois. Si le solde est négatif, alors une lettre sera

adressée au titulaire du compte.

Question : On demande de fournir un diagramme Entité-Association.

Exercice 2 : (4.5 points)

Question: Transformer le modèle entités/associations suivant en modèle relationnel.



Exercice 3 : (7points)

Soit la base de données suivante :

- Développeur (NumDev, NomDev, AdtDev, EmailDev, TelDev)
- Projet (NumProj, TitreProj, DateDeb, DateFin)
- Logiciel (CodLog, NomLog, PrixLog, #NumProj)
- Réalisation (#NumProj, #NumDev)

Les attributs soulignés sont les attributs appartenant à la clé primaire et les attributs précédés par un dièse sont des clés étrangères.

Question : Exprimez les requêtes suivantes en algèbre relationnelle :

1. Donner les numéros et les titres des projets qui ont débuté après le premier janvier 2023.
2. Afficher les noms et les adresses des développeurs qui ont travaillé sur le projet de titre "ProjetA".
3. Afficher les codes et les noms des logiciels utilisés par le développeur « Samir ».
4. Afficher les noms et les numéros des développeurs qui ont travaillés sur tous les projets.
5. Donner pour chaque développeur son nom, son mail et le nombre des projets.
6. Donner la liste des projets de développeur qui réside dans le même endroit que le développeur « Ali ». (On considère qu'Ali a un seul développeur voisin).

Exercice 4: (4.5 points)

La figure suivante donne une instance de la base « Immeuble ».

nomImmu	adresse	nbEtages	anneeConstruction	nomGérant
Koudalou	3 Rue Blanche	15	1975	Doug
Barabas	2 Allée Nikos	2	1973	Ross

La table Immeuble

nomImmu	noApp	superficie	etage
Koudalou	1	150	14
Koudalou	34	50	15
Koudalou	51	200	2
Koudalou	52	50	5
Barabas	1	250	1
Barabas	2	250	2

La table Appartement

nom	age	profession
Ross	51	Informaticien
Alice	34	Cadre
Rachel	23	Stagiaire
William	52	Acteur
Doug	34	Reinier

La table Personne

nomImmu	noApp	nomOcc	anneeArrivee
Koudalou	1	Rachel	1992
Barabas	1	Doug	1994
Barabas	2	Ross	1994
Koudalou	51	William	1996
Koudalou	34	Alice	1993

La table Occupant

Question : Pour chacune des requêtes suivantes, exprimez sa signification et donnez son résultat.

1. π (nom, age) Personne.
2. π (nomImm, noApp) (σ (superficie > 150) (Appart)).
3. π (nomGerant, superficie) (Immeuble $\bowtie$ (nomImm=nomImm) Appart).
4. π (nomImm, noApp) Appart - π (nomImm, noApp) Occupant.
5. π (nomImm) Immeuble - π (nomImm) (σ (nomOccupant='Doug') Occupant) .

BON TRAVAIL



Département des Technologies de l'Informatique

Examen

Unité d'Enseignement : POO	Classes : DS/2
Durée : 1H30	Nombre de pages : 03
Date : 04/01/2024	Heure de début : 11H
Proposé par : T.DHIFALLAH- M.BOUKHCHIM- H.GHZEL	Documents Autorisés : Non

N.B : On tiendra compte de la présentation et de la lisibilité des réponses.

Exercice

Compléter les classes nécessaires à l'exécution de ce programme tout en sachant que les classes Directeur, Ingenieur et Technicien dérivent de la classe Employe.

Résultat du programme :

si n = 1 : on obtient à l'écran : je suis l'employé Med et j'ai 29 ans.
 si n = 2 : on obtient à l'écran : je suis le technicien ali, j'ai 35 ans et je suis un cadre moyen.
 si n = 3 : on obtient à l'écran : je suis l'ingénieur Meriam, j'ai 37 ans, je suis un cadre principale et mon salaire est de 1000dt.
 si n = 5 : on obtient à l'écran : je suis le Directeur Salah, j'ai 49 ans, je suis un haut cadre et mon salaire est de

```
public class TestEmploye{
    public static void main (String[] args)
    {
        int n = 3;
        Employe p;
        switch (n)
        {
            case 1 : p = new Employe ("Med", 29); break;
            case 2 : p = new Technicien ("ali", 35, "cadre moyen");break;
            case 3 : p = new Ingenieur ("Meriam", 37, "cadre principale", 1000);break;
            case 4 : p = new Directeur ("salah", 49, "haut cadre", 3000); break;
        }
        p.affiche ();
    }
}
```

Problème

Dans le cadre de l'informatisation d'une mairie, on veut automatiser le calcul des impôts locaux.

On distingue deux catégories d'habitation : les habitations à usage professionnel et les maisons individuelles.

L'impôt se calcule différemment selon le type d'habitation. Pour cela, on définit les classes : **HabitationProfessionnelle** et **HabitationIndividuelle** et les caractéristiques communes à ces deux classes sont regroupées dans la classe **Habitation**.

On a donc un schéma de classes ou les classes **HabitationProfessionnelle** et **HabitationIndividuelle** **héritent** de la classe **Habitation**.
L'objet de ce problème est d'implémenter ce schéma d'héritage et de mettre en œuvre le mécanisme de liaison dynamique.

1) Définir la classe **Habitation** contenant :

- ❖ Les attributs suivants :
 - propriétaire du type chaîne de caractères et qui correspond au nom du propriétaire,
 - adresse du type chaîne de caractères et qui correspond à l'adresse de l'habitation.
 - surface du type double et qui correspond à la surface de la classe **Habitation**.
- ❖ Un constructeur paramétré permettant d'initialiser une instance de la classe **Habitation**.
- ❖ Les méthodes :
 - **GetType()** : qui renvoie le type de l'objet courant.
 - **double Impot()** : qui permet de calculer le montant de l'impôt que doit payer le propriétaire de l'habitation à raison de 0,5dinars par m².
 - **public String toString()** : qui permet de retourner le type ainsi que l'état de l'instance, c'est-à-dire les valeurs de ses attributs.
 - **void Affiche()**.

2) Définir la classe **HabitationIndividuelle** qui hérite de la classe **Habitation** et contenant :

- ❖ Les attributs suivants :
 - NbPieces de type entier,
 - Piscine de type booléen.
- ❖ Un constructeur paramétré permettant d'initialiser une instance de la classe **HabitationIndividuelle**.
- ❖ Redéfinir les méthodes :
 - **GetType()** : qui renvoie le type de l'objet courant.
 - **double Impot()** : Le calcul de l'impôt d'une maison individuelle est différent de celui d'une habitation, il se calcule en fonction de la surface habitable (même tarification que pour une habitation), du nombre de pièces et de la présence ou non d'une piscine. On compte supplémentaire 2dinars/pièce et 50dinars en cas de présence d'une piscine.
 - **public String toString()**.
 - **void Affiche()**.

3) Définir la classe **HabitationProfessionnelle** qui hérite de la classe **Habitation** et contenant :

- ❖ L'attribut suivant :
 - NbEmployes de type entier.
- ❖ Un constructeur paramétré permettant d'initialiser une instance de la classe **HabitationProfessionnelle**.
- ❖ Redéfinir les méthodes :
 - **GetType()** : qui renvoie le type de l'objet courant.
 - **double Impot()** : Le calcul de l'impôt d'une habitation à usage professionnel est également différent de celui d'une habitation. Il se calcule en fonction de la surface occupée par le bâtiment (même tarification que pour une habitation) et du nombre d'employés travaillant dans l'entreprise. On compte 100dinars supplémentaire par tranche de 10 employés (on arrondit à la tranche la plus petite).
 - **public String toString()**.
 - **void Affiche()**.

Remarque : les constructeurs et les méthodes `Impot()` et `toString()` des deux classes dérivées devront obligatoirement utiliser les méthodes appropriées de la classe parente.

- 4) Créer une classe **Collection** qui utilise la classe **Habitation**. Cette classe permet de gérer les habitations d'une commune. Cette classe est caractérisée par les attributs suivants :
- `tab` : représente un tableau polymorphe d'habitations.
 - `Taille` : représente le nombre d'habitations de la commune.
 - `Max` : représente le nombre maximal d'habitations de la commune.
 - Ainsi que les méthodes suivantes :
 - Un constructeur qui crée une collection ayant un nombre maximum d'habitations indiquée en paramètre et initialise la taille de la collection.
 - `void ajouterHabitation(Habitation)` qui ajoute une habitation à toutes les
 - `void Afficher_collection()` qui affiche les informations relatives à l'impôt habitations de la collection (n'oubliez pas d'afficher aussi la valeur de l'impôt accordée à chaque habitation)
 - `Collection Liste_HabIndiv()` : C'est une méthode qui retourne une collection formée seulement des habitations individuelles.
 - `Collection Liste_HabProf()` : C'est une méthode qui retourne une collection formée seulement des habitations professionnelles.
- 5) Créer une classe `testcollection` contenant la méthode **main()** dans laquelle vous allez :
- Créer une collection d'habitations C1
 - Remplir son tableau par 5 instances des classes `HabitationIndividuelle` et `HabitationProfessionnelle`.
 - Pour chaque élément du tableau, afficher ses caractéristiques ainsi que la valeur de son impôt.
 - Créer une collection C2 et la remplir par les habitations individuelles de C1
 - L'afficher à l'écran
 - Créer une collection C3 et la remplir par les habitations professionnelles de C1
 - L'afficher à l'écran

BON TRAVAIL...



Département des Technologies de l'Informatique

Examen Final

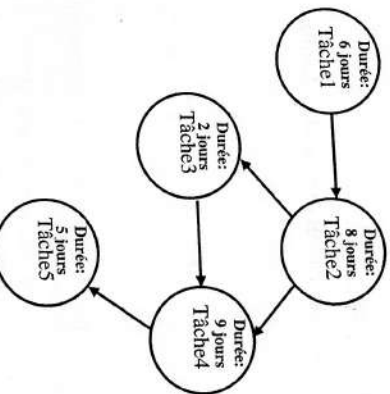
Unité d'Enseignement : Programmation Avancée	Classes : D.S.I,2
Durée : 01 Heure 30 minute	Nombre de pages : 2
Date : 09/01/2024	Heure de début : 13H00
Proposé par : Mme Thouraya DHIFALLAH, Mr Moez ARFAOUI, Ali Mensi	Documents Autorisés : Non

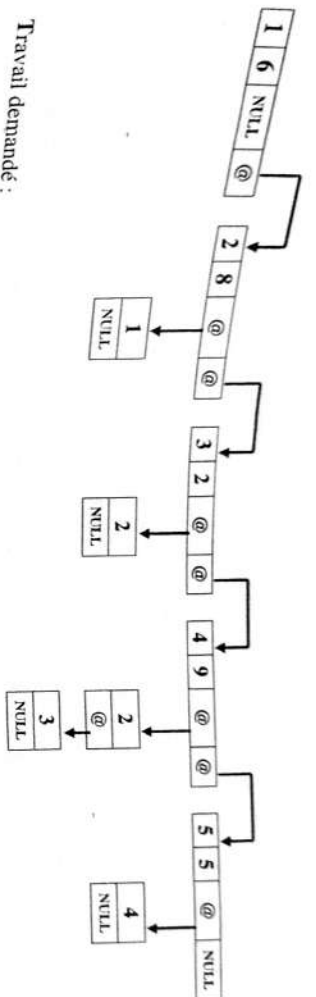
NB :

- Le détail des réponses à tous les exercices doit figurer dans la feuille de réponse.
- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

Problème :

La planification d'un projet correspond à l'organisation des tâches à réaliser sur une période donnée. L'objectif de la planification est d'avoir une vision claire de son projet et de le réaliser dans un minimum de temps.
A partir de ce besoin, on souhaite écrire un programme C qui prend en charge les tâches d'un projet ainsi que leurs dépendances afin d'aboutir à une meilleure organisation chronologique et ce en utilisant la structure chaînée **graphe**.
Sachant qu'une tâche est identifiée par un numéro et caractérisée par une durée : Ci-joint un exemple illustrant la prise en charge des tâches dans la mémoire.





Travail demandé :

1. Définir les structures nécessaires pour manipuler un graphe de tâches et de leurs dépendances.
2. Implémenter le sous-programme `graphe Init()` qui permet d'initialiser le graphe à NULL.
3. Implémenter une fonction `graphe ajout_sommet(graphe g, int t, int n)` qui permet d'insérer une nouvelle tâche identifiée par `t` et qui dure `n` jours à un graphe passé en paramètre.
4. Implémenter une fonction `graphe ajout_lien(graphe g, int num1, int num2)` qui permet d'ajouter un lien de dépendance ordonné entre la tâche1 identifiée par `num1` et la tâche2 identifiée par `num2` (tâche1 dépend de tâche 2).
5. Implémenter une fonction `graphe creer_graphe()` qui permet de remplir un graphe par les différentes tâches dont chacune avec la liste de ses liens de dépendance et de le retourner. (Les informations de chaque tâche doit être saisies à partir du clavier).
5. Implémenter une procédure `void affiche_graphe(graphe g)` permet d'afficher l'ensemble des tâches ainsi que leurs dépendances à l'écran.
7. Implémenter la fonction `int duree_projet(graphe g)` qui permet de calculer et de retourner la durée du projet en nombre de jours.
8. Créer le programme principal qui permet de :
 - Créer un graphe modélisant les tâches dépendantes d'un projet initialisé à vide
 - Le remplir à partir du clavier et l'afficher à l'écran
 - Calculer et afficher la durée du projet modélisé par le graphe.

Exercice : Arbres Binaires de Recherche (ABR)

1. Lesquels de ces arbres ne sont pas des arbres binaires de recherche ? Justifier brièvement.



2. Dessiner l'ABR obtenu par création en insérant successivement les éléments suivants dans cet ordre : 45, 24, 0, 16, 10, 36, 44, 64, 88, 22, 50.
3. Implémenter un sous-programme `recursif void AffichDecroissant(ABR r)` qui permet d'afficher les éléments de l'arbre binaire de recherche de racine `r` dans l'ordre décroissant.

Bon Travail



DS

Unité d'Enseignement : Bases de Données		Classes : DSI 2
Durée : 1 Heure		Nombre de pages : 1
Date : 14/11/2019		Heure de début : 9H00
Proposé par : Ons Ben Rejeb et Mbarka Ben Yousef		Documents Autorisés : NON

NB :

- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

Questions de cours : (6 pts)

1. Définir brièvement les trois niveaux de l'architecture standard ANSI/SPARC.
2. Donner un exemple d'une association réflexive.
3. Donner un exemple d'une association porteuse d'un attribut et de cardinalité (0, N) – (0, N).
4. Donner un exemple d'une association de cardinalité (1, 1) – (1, N).

Exercice : (14 pts)

Il s'agit d'établir le schéma conceptuel d'une base de données pour la gestion des formations d'un institut privé. Un cours est caractérisé par un numéro de cours (NO_COURS), un libellé (LIBELLE), une durée en heures (DUREE) et un type (TYPE). Un cours peut faire l'objet dans l'année de plusieurs sessions identiques. Une session est caractérisée par un numéro (NO_SES), une date de début (DATE_DEB) et un prix (PRIX). Une session est le plus souvent assurée par plusieurs animateurs et est placée sous la responsabilité d'un animateur principal. Un animateur peut intervenir dans plusieurs sessions au cours de l'année. On désire mémoriser le nombre d'heures (NB_H) effectué par un animateur pour chaque session. Un animateur est caractérisé par un numéro (NO_ANI), un nom (NOM_A) et une adresse (ADR_A). Chaque session est suivie par un certain nombre de participants. Un participant est une personne indépendante ou un employé d'une entreprise cliente. Un participant est caractérisé par un numéro (NO_PAR), un nom (NOM_P) et une adresse (ADR_P). Dans le cas d'un employé, on enregistre le nom (NOM_EN) et l'adresse de l'entreprise (ADR_EN).

Etablir un schéma conceptuel (Entité/Association) pour cette base de données.

Ben Travail



EXAMEN

Unité d'Enseignement : <i>Conception Objet</i>	Date : <i>02 Janvier 2019</i>
Elément Constitutif (ECUE) : <i>Modélisation Objet UML</i>	Classes : <i>DSI 21</i>
Durée : <i>1h 30 minutes</i>	Nombre de pages : <i>2</i>
Proposé par : <i>Yosra KASSIS</i>	Documents Autorisés : <i>Néant</i>

NB :

- Le détail des réponses à tous les exercices doit figurer dans la feuille de réponse.
- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

Exercice 1 : ANALYSE DES BESOINS D'UN COMMISSARIAT DE POLICE

Un commissariat de police fonctionne de la façon suivante. Le standard reçoit les appels téléphoniques de plaignants ou de témoins qu'il doit alors aiguiller vers la division (ou service) adéquate. Auparavant, il doit enregistrer sur ordinateur chaque appel (date, coordonnées du plaignant ou témoin, type de plainte ou témoignage). Les plaignants doivent ensuite venir vérifier puis signer leur plainte à l'accueil du commissariat. Les témoins ne sont convoqués pour compléments d'informations que si une plainte a été déposée.

Il existe essentiellement trois divisions au commissariat :

- ✓ La division des délits mineurs (vols, violence, etc.)
- ✓ La division des trafics de stupéfiants
- ✓ La division des crimes

Le commissaire répartit les missions de la journée sur les ressources dont il dispose (inspecteurs, policiers, véhicules, chiens). Une mission traite un ensemble de délits qui ont eu lieu dans la même zone géographique. Il définit l'ordre des étapes puis affecte à la mission au moins un véhicule et deux policiers. Pour toute plainte/témoignage qui relève des deux dernières divisions, au moins un inspecteur est affecté à l'affaire. Les délits liés à un trafic de stupéfiants requièrent aussi l'attribution d'un ou deux chiens. Le secrétariat du commissariat doit alors enregistrer la mission et son heure de départ.

Les policiers partis en mission avec les bordereaux décrivant chaque plainte/témoignage doivent indiquer en temps réel l'état de leur mission (sur terminal portable) : les arrêts et les départs aux différentes étapes ainsi que les incidents occasionnels qu'ils peuvent rencontrer (panne, retard...). Si, au-delà d'une heure, le système n'enregistre aucun suivi de cette mission, il doit alerter le commissaire immédiatement. Celui-ci crée alors une mission de renfort dont la seule étape est la dernière rapportée par la mission qui s'est déroulée de façon anormale.

Les inspecteurs de la division des crimes doivent mener leur enquête en étroite collaboration avec un juge du tribunal d'instance. Le service des archives du commissariat les aide dans leur enquête en leur fournissant toutes les informations qu'ils trouvent en relation avec leur affaire. Une fois l'enquête terminée, les policiers, munis d'un mandat d'arrêt du juge affecté à l'affaire, procèdent à l'arrestation du ou des coupables désignés. Chaque arrestation doit alors faire l'objet d'un rapport (date, coordonnées des coupables, lieu) que les policiers doivent enregistrer sur ordinateur.

Travail demandé :

1. Donner le diagramme des cas d'utilisation qui décrit le fonctionnement de ce commissariat de police.
2. Elaborez le diagramme de classes correspondant.
3. Elaborez le diagramme de séquence objet du scénario « *Création de mission pour division des trafics de stupéfiants* » du cas « *Répartir les missions* ».

Bonne chance

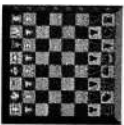
EXAMEN

Unité d'Enseignement : Conception Objet		Date : 02 Janvier 2019 à 11h
Element Constituant (ECUE) : Modélisation Objet UML		Classes : DSI 22
Durée : 1h Heure 30 minutes		Nombre de pages : 3
Proposé par : Nour El. AOULINI		Documents Autorisés : Néant

NB :

Le détail des réponses doit figurer sur la feuille **BLANCHE** de réponse.

Etude de cas : JEU D'ECHEC



Dans cet exercice, il vous est proposé de modéliser le jeu d'échec. Selon l'interview avec l'expert métier, les règles du jeu sont les suivantes :

- ✓ Le jeu d'échecs se joue à deux joueurs sur un échiquier carré composé de **64** cases, alternativement **noires** et **blanches**. L'échiquier est caractérisé par sa matière et sa taille.
- ✓ Chaque case possède une rangée (entre **1** et **8**) et une colonne (entre **a** et **h**).
- ✓ Il ne peut y avoir qu'une pièce au maximum sur une case donnée. Une pièce est soit sur une case, soit hors du jeu (prise).
- ✓ Chaque pièce est d'une couleur **noire** ou **blanche**. Les pièces peuvent se déplacer mais leurs positions initiales sont représentées sur la figure précédente.
- ✓ Une pièce peut être un roi, une dame, un tour, un fou, un cavalier ou un pion.
- ✓ Chaque joueur, de rôle **noir** ou **blanc**, est caractérisé par son nom, son prénom ainsi que son classement et possède initialement huit pions, un roi, une dame, deux tours, deux fous et deux cavaliers. Lorsque les pions arrivent en huitième rangée, ils doivent se transformer obligatoirement en toute autre pièce sauf en roi, ainsi un joueur peut posséder jusqu'à neuf dames, dix tours, cavaliers ou fous.
- ✓ A chaque tour, un joueur bouge une pièce de sa case. On ne peut ni passer son tour, ni jouer deux fois de suite. Une partie est donc formée d'une suite ordonnée de coups et elle aboutit enfin à un résultat.
- ✓ Il faut considérer la contrainte suivante : La couleur du coup joué doit être la même que celle de la pièce à bouger.
- ✓ A noter que dans cet exercice on ne s'intéresse pas aux pièces prises (hors-jeu).

Travail demandé :

1. Modéliser, par un diagramme de classes, le matériel utilisé par les joueurs et le déroulement d'une partie.
2. Proposer une structuration en package de votre diagramme de classes.
3. Compléter le **diagramme d'états** ci-dessous correspondant à une partie d'échec, sachant que :
 - ✓ Les **blancs** commencent toujours la partie.
 - ✓ Lorsque l'un joueur abandonne, son adversaire gagne la partie.
 - ✓ La partie se termine par l'un de ces états finaux :
 - **Gain du blanc (1-0)** : Lorsque le joueur blanc joue le coup « **échec et mat** »
 - **Gain du noir (0-1)** : Lorsque le joueur noir joue le coup « **échec et mat** »
 - **Partie nulle (1/2-1/2)** : Cet état se produit suite à un de ces événements :
 - « ***Pat*** » (l'un des joueurs ne peut plus exécuter aucun coup)
 - « ***Triple répétition de position*** » (une position identique se produit pour la troisième fois)
 - « ***La nulle mutuelle*** » (un joueur peut proposer la nulle à son adversaire pendant la partie. Si ce dernier accepte, la partie est nulle)

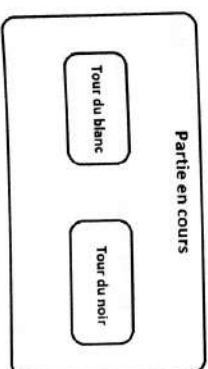


Diagramme d'état

BON TRAVAIL

Département TI
Examen Techniques de Communication

Durée : 1,5h

Groupes : DSI 21.22 et RS/21

Enseignants: L. Ben Abdallah,
Nom :

Prénom : Groupe:.....

Exercice n° 1 : (13pts)

Un entretien de recrutement entre Najes Mrad (28 ans, titulaire d'un BTS Action commerciale) et Radouane Mhamdi (42 ans, adjoint au chef du personnel) Najes. entre dans le bureau de Radouane et envoie simultanément une multitude d'informations : tailleur bien coupé, coiffure soignée, maquillage discret : femme distinguée, voix posée et réponse claire...

Radouane est assis derrière son bureau, costume cravate. Attitude bienveillante : il recherche un(e) collaborateur (trice).

Ils vont évoluer tout au long de l'entretien : il s'établit entre eux une relation qui touche au rapport de place et à l'image que chacun veut donner (Najes au début a une position basse, puis la relation devient plus symétrique, avec la possibilité d'avoir une position haute si elle répond aux critères du recruteur).

Dialogues de l'entretien :

1. Elle : Bonjour Monsieur Mhamdi (*elle tend la main, regarde dans les yeux, position assurée, buste droit*)
2. Lui : Bonjour Mademoiselle Mrad. Veuillez vous asseoir, s'il vous plaît (il lui indique une chaise)
3. Elle : je vous remercie (*s'assoit en face, mains sur la table, jambes serrées, repliées du même côté*)
4. Lui : Pouvons-nous commencer, Mlle?
5. Elle : Oui Monsieur
6. Lui : Je vous ai convoquée, pour approfondir quelques éléments que je trouvais intéressants à la lecture de votre CV. Parlez moi de vous...
7. Elle : J'exerce depuis 6 ans un emploi de commerciale dans la société Solaconfi, spécialisée dans la distribution de confiseries. Mon rôle est d'implanter nos produits auprès des détaillants. Je veux évoluer et je pense que mon profil convient parfaitement au poste de chef de secteur que vous proposez.
8. Lui : Mais il ne s'agit pas d'implanter auprès des détaillants, plutôt de négocier avec la grande distribution... Pensez-vous qu'une femme saura s'imposer dans ce milieu hostile ?
9. Elle : Oui. Les femmes possèdent des atouts que les hommes n'ont pas. Elles peuvent utiliser bien des arguments, notamment dans un milieu masculin.
10. Lui : - 1er cas : il la recadre et oriente la discussion sur un plan purement professionnel (il recherche un collaborateur efficace, dans l'intérêt de la société)
- 2ème cas : il entre dans le jeu de la séduction et teste son intégrité et ses limites.

1/ Dégagez les éléments de cette situation de communication (émetteur, récepteur, message, canal, code, contexte,...) :

2/ Dégagez les éléments non verbaux de la communication pour chacun des interlocuteurs.

Département TI
D.S. Techniques de Communication

Durée : 1h

Groupes : DSI 21.22 et RSI/21

Enseignants: L. Ben Abdallah,

Nom :

Prénom :

Groupe:.....

Exercice n° 1 :

Mise en situation : Wided travaille pour M. Paul Lebrun. À 9 h 15, la veille du congé

M. Lebrun lui remet des états financiers ainsi que deux adresses. Sur l'entrefaite, le téléphone sonne et le patron accélère le débit de son message; Wided ne peut répondre sachant que son patron ne dispose que de peu de temps : il est déjà attendu à une réunion.

Scénario

M. Lebrun : Faites parvenir ce document à Mme Aubin pour demain, Vendredi saint. S'il n'y a pas de messages demain, informez-vous à quelle heure le document pourrait lui être livré aujourd'hui.

Le téléphone sonne de nouveau. Sans interrompre son patron, Wided met le répondeur en fonction.

M. Lebrun : Avant 16 h 30, utilisez l'adresse du bureau de Mme Aubin; après 16 h 30, celle de son domicile. Prenant pour acquis que Wided a tout compris, M. Lebrun se dirige vers la salle de réunion. Le mardi suivant, Mme Aubin téléphone à M. Lebrun afin de l'aviser que le document a été livré à son bureau le jeudi précédent à 17 h 10. Elle était déjà partie...

N'ayant reçu aucune information supplémentaire, ni verbale ni écrite, et connaissant très peu la ville où se trouvent les deux adresses transmises par son patron, Wided a cru que l'adresse sur le boulevard était celle du bureau, et l'adresse mentionnant une avenue, celle du domicile de Mme Aubin.

TAF

1/ **Dégagez les éléments de la communication :**

2/ **Dans notre mise en situation, il y a des « bruits », c'est-à-dire des éléments qui n'ont pas favorisé la communication. Quels sont ces « bruits » ?**

Ne Rien Ecrire Ici

Exercice n°2 :

1/ Dans chacune des phrases, soulignez en BLEU l'émetteur, en VERT le récepteur, en NOIR le message et indiquez le code et le canal.

Le professeur de français sera absent demain annonce un surveillant aux élèves.

Le canal :

Le code :

Paul écrit à son ami qu'il lui rendra visite le mois prochain.

Le canal :

Le code :

Luc demande l'heure à son voisin.

Le canal :

Le code :

Monsieur Dupont écoute le président de la république annonçant son programme politique à la télévision.

Le canal :

Le code :

Bon travail

Devoir Surveiller

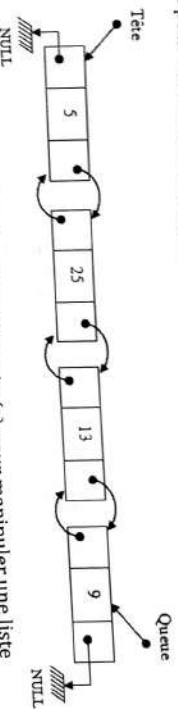
Unité d'Enseignement : <i>Programmation avancée</i>	Parcours : <i>DSI</i>
Elément Constitutif (ECUE) : <i>Unité Optionnelle</i>	Classes : <i>DSI 21 , DSI22</i>
Durée : <i>1h Heure</i>	Nombre de pages : <i>02</i>
Proposé par : <i>Mme Thiouraya D. Mr Moeza</i>	Documents Autorisés : <i>NON</i>

NB :

- Le détail des réponses à tous les exercices doit figurer dans la feuille de réponse.
- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

Exercice 1

Soit l'exemple ci-dessus d'une liste chaînée double. Pour chacune des questions suivantes, on vous demande de développer le sous-programme adéquat pour réaliser le traitement.

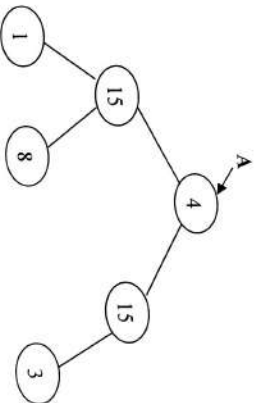


- 1) Définir la/les structure(s) de données nécessaire(s) pour manipuler une liste doublement chaînée.
- 2) Ecrire une fonction **CreerLbi** qui permet de créer une liste doublement chaînée vide.
- 3) Ecrire une procédure **AjouterTete** qui permet d'ajouter un élément à la tête d'une liste doublement chaînée.
- 4) Ecrire une procédure **AjouterQueue** qui permet d'ajouter un élément à la queue d'une liste doublement chaînée.
- 5) Ecrire une procédure **InsererApres** qui permet d'insérer un élément après une position (adresse) dans une liste doublement chaînée.
- 6) Réécrire la procédure **AjouterQueue** en utilisant la procédure **InsererApres**.
- 7) Ecrire la fonction **Rechercher** qui permet de vérifier si un élément **x** existe dans la liste. Elle renvoie l'adresse de la valeur s'il existe, si non elle renvoie **NULL**.
- 8) Ecrire une procédure **SupprimerVal** (en utilisant la procédure **Rechercher**) qui permet de supprimer un élément ayant une valeur donnée d'une liste doublement chaînée.
- 9) Ecrire une procédure **SupprimerQueue** qui permet de supprimer la queue d'une liste doublement chaînée.

Exercice 2

Soit l'exemple, ci-dessous, d'un arbre binaire
On vous demande de :

- 1) Définir la structure de donnée qui permet de représenter un nœud de l'arbre
- Développer les sous-programmes permettant de réaliser les traitements suivants:
- 2) Calculer la somme des valeurs de cet arbre
- 3) Chercher le nombre des nœuds totaux
- 4) Calculer la hauteur de l'arbre



Exemple d'arbre binaire (d'entiers)

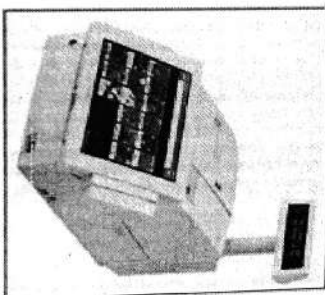
Devoir Surveillé

Parcours : Développement des Systèmes d'Information	
Unité d'Enseignement : Conception Objet	Classes : DSI 21, 22
Élément Constitutif (ECUE) : Modélisation Objet UML	Nombre de pages : 2
Durée : 1h Heure	Documents Autorisés : Néant
Proposé par : Nour EL AOULI, Yosra KASSIS	

NB :

- Le détail des réponses à tous les exercices doit figurer dans la feuille de réponse.
- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

Exercice 1 : ÉTUDE D'UN TERMINAL POINT DE VENTE (TPV)



Cet exercice concerne un système simplifié de caisse enregistreuse de supermarché. Le déroulement normal d'utilisation de la caisse est le suivant :

- ✓ Un client arrive à la caisse avec des articles à payer.
- ✓ Le caissier enregistre le numéro d'identification (CPU) de chaque article, ainsi que la quantité si elle est supérieure à un.
- ✓ La caisse affiche le prix de chaque article et son libellé.
- ✓ Lorsque tous les achats sont enregistrés, le caissier signale la fin de la vente.
- ✓ La caisse affiche le total des achats.
- ✓ Le client choisit son mode de paiement :
 - numéraire : le caissier encaisse l'argent reçu, la caisse indique la monnaie à rendre au client ;

- chèque : le caissier vérifie la solvabilité du client en transmettant une requête à un centre d'autorisation via la caisse ;
- carte de crédit : un terminal bancaire fait partie de la caisse. Il transmet une demande d'autorisation à un centre d'autorisation en fonction du type de la carte.

- La caisse enregistre la vente et imprime un ticket.

✓ Le caissier donne le ticket de caisse au client. Après la saisie des articles, le client peut présenter au caissier des coupons de réduction pour certains articles. Lorsque le paiement est terminé, la caisse transmet les informations sur le nombre d'articles vendus au système de gestion de stocks. Tous les matins, le responsable du magasin initialise les caisses pour la journée.

Travail demandé

Établir le diagramme des cas d'utilisation.

Exercice 2 : Application Web genre réseau social

Les stagiaires d'un établissement de formation désirent développer une application Web genre réseau social. L'application permet aux membres inscrits de créer un réseau d'amis et d'échanger des contenus.

Pour devenir membre de cette application, un internaute doit s'inscrire lors de sa visite du site web. Lorsque l'administrateur valide l'inscription, l'internaute devient membre ; il peut alors se connecter pour réaliser les opérations suivantes : chercher un membre, inviter un membre d'être son ami, accepter un autre membre comme ami, refuser une invitation d'amitié, retirer un ami de sa liste d'amis. Un membre peut également publier des contenus.

Un modérateur consulte les contenus publiés par les membres et peut avertir, par un message, un membre qui publie du contenu non conforme au règlement de l'application ; après 3 avertissements, l'administrateur supprime le compte de ce membre.

Un membre est identifié par une adresse e-mail, un nom et un prénom. Un membre peut avoir plusieurs amis qui sont aussi membres de l'application. Un membre peut effectuer une publication de contenu à une date donnée. Une publication concerne une ressource à publier ; une ressource possède un nom et peut être soit un message, soit une photo/vidéo soit un lien vers une page web. Une photo/vidéo a en plus du nom, une taille en Mo. Un lien vers une page web a une adresse (URL).

Les amis d'un membre sont organisés en types d'amis : les amis peuvent être de type « Ami normal », « Amis proches », « connaissances » ou « Famille ». Une publication d'une ressource par un membre est autorisée à être consultée seulement par un type d'amis donné ; par exemples une photo est partagée uniquement avec le type d'amis « Amis proches ».

Travail demandé

Établir le diagramme de cas d'utilisation.

ISET de ZAGHOUAN Département des Technologies de l'Informatique		
République Tunisienne Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de la technologie Direction Générale des Etudes Technologiques		
Classes DSI 21-22 Le 15/11/2019 SI Documents Non Autorisé	Devoir surveillé Programmation Orienté objet	Semestre 1 (A.U. 2019-2020) Enseignants : M. Boukchim – A. Mechmech

Question de Cours

1. A quoi sert un constructeur dans une classe ?
2. Quelle est la différence entre une classe et un objet ?
3. Citer les 3 types de visibilité qui existe en JAVA et expliquer la différence en eux.

Exercice 1 :

Les employés d'une société sont caractérisés par leur nom (chaîne de caractères), leur salaire (réel) et leur matricule (entier).

Q1 : Complétez la classe *Employe* ci-dessous en précisant ces attributs, un constructeur qui initialise la matricule et le nom de l'employé et les deux méthodes :

- Une méthode *afficher()* qui permet d'afficher les caractéristiques d'un employé selon le format de message suivant :

« L'employé [nom] de matricule [matricule] est payé [salaire] DT chaque mois. »

- Et une deuxième méthode *augmenter()* qui permet d'augmenter le salaire de l'employé de 5%.

```

class Employe {
    // Attributs caractérisant un employé

    // Constructeur permettant de créer un employé.
    ..... (int mat, String nome) {
        matricule = mat;
        nom = nome;
        salaire = 6000;
        System.out.println("Un employé vient d'être crée : ");
        afficher();
    }
  }

```

```
void afficher() {  
    // Afficher les caractéristiques d'un employé  
}  
  
void augmenter () {  
    // augmenter le salaire d'un employé de 5 %  
}
```

Q2 : Utilisez le programme suivant pour tester votre classe Employe

```
public class TestEmploye {  
    public static void main (String[] argv) {  
        /* Création d'une instance de la classe Employe qu'on nommera Dup de  
        matricule 1 et de nom Dupont */  
        .....  
    }  
}
```

// Augmenter Dup

.....

// Réafficher les caractéristiques de Dup

.....

}

}

Exercice 2 :

Une entreprise souhaite informatiser la gestion de son personnel qui est constitué principalement de cadres, techniciens et administrateurs système.

Le responsable du service personnel a décidé de gérer toutes ses activités par des programmes en langage orienté objet (java). Une personne est représentée par son code CIN, son nom et prénom, son âge et son adresse.

Q1 : Définir la classe Personne qui permet de :

- Déclarer les attributs d'une personne.
- Un constructeur qui permet d'initialiser les attributs de la classe.
- Créer une méthode qui permet d'afficher les attributs d'une personne.

Q2 : Ajouter les méthodes nécessaires pour que ces attributs deviennent accessibles en dehors de la classe.

Un cadre est une personne caractérisé aussi par son ancienneté et sa fonction dans l'entreprise.

Q3 : Donner la définition de la classe Cadre.

Q4 : Ajouter à la classe Cadre une méthode qui incrémente (+1) l'ancienneté d'un cadre.

Examen

Unité d'Enseignement : <i>Programmation avancée</i>	
Elément Constitutif (ECUE) : Unité Optionnelle	Parcours : DSI
Durée : 1h : 30	Classes : DSI 21 , DSI22
Proposé par : Mr Moez, A. Mme Thouraya, D.	Nombre de pages : 02
	Documents Autorisés : NON

NB :

- Le détail des réponses à tous les exercices doit figurer dans la feuille de réponse.
- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

Exercice 1 (10 ps) Fichier et arbre

- 1) Rappeler la propriété fondamentale d'un arbre binaire de recherche (ABR)
- 2) On se propose de trier un fichier contenant que des nombres entiers dans l'ordre croissant. Le résultat sera placé dans un second fichier résultat.
 - a) Pour cela, on vous demande de transférer le fichier initial dans un ABR. Développer le sous programme nécessaire nommé `Transfert_Fichier_arbre(...)`
 - b) Quel est le parcours d'un arbre binaire qui vous permet d'afficher son contenu dans l'ordre croissant.
Développer alors le sous programme qui permet d'afficher les éléments d'un ABR dans le sens croissant.
 - c) Développer le sous programme nommé `Transfert_arbre_fichier(...)` qui permet de transférer le contenu de l'arbre dans un fichier.
- 3) Ecrire le programme principal qui permet :
 - D'accepter au clavier le *chemin* du fichier source à trier et celui du fichier résultat.
 - D'appeler les sous programmes nécessaires pour trier le fichier source en passant par l'arbre (ABR).



Exercice 2 (8 pts) Liste et arbre

Afin de simplifier le tri d'une liste doublement chaînée, on se propose de passer par les ABR. Pour cela on vous demande de :

- 1) Rappeler la /les structure(s) de donnée pour représenter une liste doublement chaînée
- 2) Développer un sous programme `Transfert_List_ABR(...)` , permettant de transférer le contenu de la liste vers un ABR
- 3) Développer un sous programme `Transfert_ABR_Liste(...)` , permettant de transférer le contenu d'un ABR vers une liste.
- 4) Développer un sous programme `Est_triee (...)` permettant de vérifier si une liste est triée ou non.
- 5) Développer un sous programme `Affiche_Liste(...)` permettant d'afficher le contenu de la liste sur l'écran.
- 6) Ecrire le programme principal qui permet de tester les sous programme précédents :
 - Créer et remplir une liste doublement chaînée d'entiers
 - Tester si cette liste est triée ou non. Dans le cas contraire, appeler les sous programmes déjà développés pour la trier dans la même liste
 - L'afficher après l'avoir triée.

Bon travail ...

EXAMEN

Unité d'Enseignement : Bases de données		Classes : DS12
Durée : 1 Heure 30 minutes		Nombre de pages : 3
Date : 06/01/2020		Heure de début : 11H00
Proposé par : Mbarika Ben Youssef et Ons Ben Rejeb		Documents Autorisés : NON

NB :

- Il vous est demandé d'apporter un soin particulier à la présentation de votre copie.

Exercice 1 : (7 pts)

Enoncé :

On s'intéresse à la vente de produits dans une pharmacie. Un produit est caractérisé par un nom de produit, une classe pharmaceutique, un nom de fabricant, un numéro de lot, une date d'expiration, et un type de remboursement (attribué par la sécurité sociale). Chaque produit est déterminé par son nom et son numéro de lot. Un produit peut être remboursable ou non remboursable. Dans le premier cas, le pourcentage du remboursement dépend de la classe pharmaceutique du produit. Le prix d'un produit n'est pas fixé, il est varié selon les périodes (promotions, augmentation de prix des fabricants, etc.). L'achat d'un produit remboursable nécessite une ordonnance d'un médecin. Un médecin est caractérisé par un nom, une adresse, un numéro d'agrément et une spécialité. Sur une ordonnance, on voit les informations concernant le médecin et le patient, la date de consultation, et une liste de médicaments avec les prescriptions de traitement. La pharmacie garde une copie de chaque ordonnance à laquelle il attribue un numéro unique. D'après la liste de produits sur l'ordonnance, une préparatrice (ou un préparateur) de la pharmacie cherche les produits et établit une facture. Chaque facture a une date, un numéro d'identification et une liste de produits avec leur quantité. Si une facture a des produits remboursables, alors on distingue la somme à payer par le patient (le client) lui-même, la somme à rembourser par la sécurité sociale et la somme à rembourser par la mutuelle. Un client est caractérisé par un nom, une adresse, un numéro de téléphone. S'il bénéficie de la sécurité sociale, alors il en a un numéro d'identification, un numéro du centre de gestion de remboursements de la sécurité sociale. Un client peut avoir au

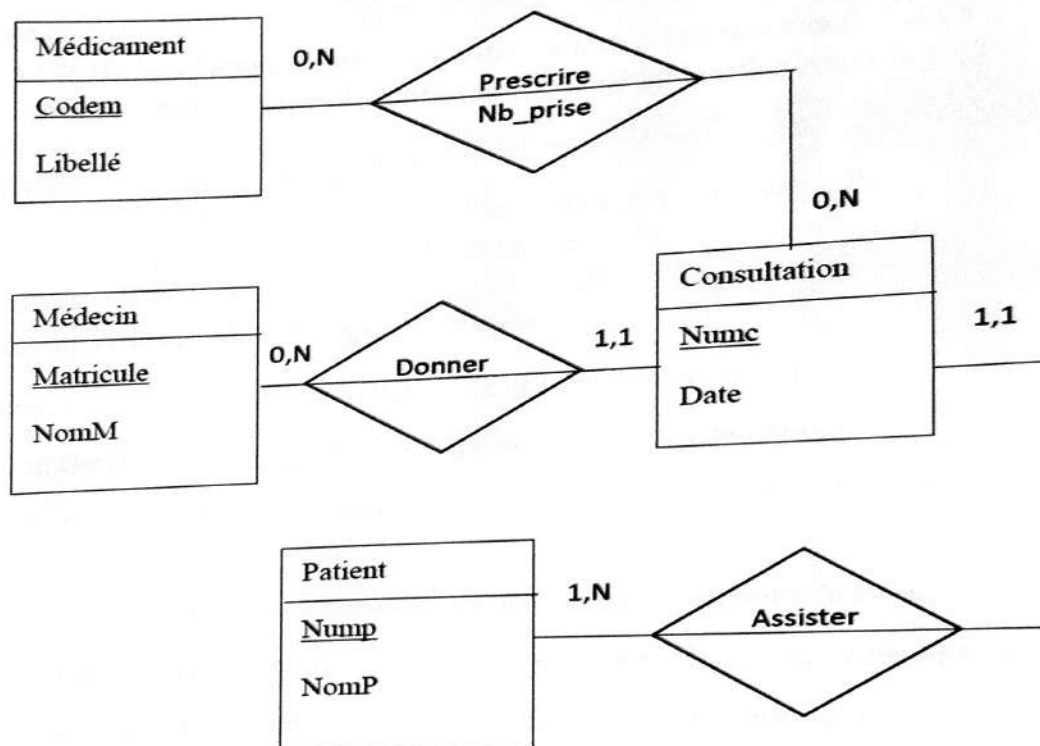
plus une seule mutuelle. Chaque mutuelle a un nom, une adresse du centre local de gestion de remboursements, dépendant de patients. Il est de même pour les centres de gestion de remboursements de la sécurité sociale.

Question :

Construire le modèle Entité-Association relatif à la gestion des produits dans une pharmacie.

Exercice 2 : (4 pts)

Question : Transformer le modèle entité/association suivant en modèle relationnel :



Exercice 3 : (4 pts)

Soient les trois relations R1, R2 et R3 :

R1	A	B
	a1	b1
	a1	b2
	a1	b3
	a3	b1
	a3	b3

R2	A	B
	a1	b1
	a2	b2
	a3	b1

R3	X	Y	Z
	x1	y1	a1
	x2	y3	a5
	x1	y1	a3

1. Déterminer l'union, l'intersection et la différence entre R1 et R2.
2. Déterminer les relations R4, R5, R6, R7 et R8 comme suit, sachant que R1.A, R2.A et R3.Z ont le même domaine :
 - a) $R4 = \pi(X, Y)R3$
 - b) $R5 = R2 \times R3$
 - c) $R6 = \pi(B, X, Y)(R2 \bowtie R3)$
 - d) $R7 = R2 \div \pi(A) R1$
 - e) $R8 = \sigma(X=x1')(R1 \bowtie R3)$

Exercice 4: (5 pts)

Enoncé :

L'agence de voyages **Tunisia Tours** organise des voyages avec des visites d'endroits touristiques. Le schéma relationnel relatif à son système d'information est décrit par les relations suivantes :

Client (NoClient, NomClient, Adresse)

Voyages (NoVoyage, VilleDépart, VilleArrivée, DateDépart, DateRetour, Prix)

Inscription (NoVoyage#, NoClient#, DateInscription)

Visite (NoVisite, Endroit)

Programmes (NoVisite#, NoVoyage#, DateVisite)

Question : Exprimer, en Algèbre Relationnelle, les requêtes suivantes :

- a) Quels sont les voyages (numéro) qui ont coûté moins de 2000Dt ?
- b) Quels sont les clients (nom et adresse) qui se sont inscrits à des voyages pendant l'année 2014.
- c) Quels sont les endroits programmés pour être visités au cours du voyage V0105 ?
- d) Quels sont les clients (nom et adresse) qui ont visité la **Sagrada Familia** (endroit) ?
- e) Quels sont les clients (nom et adresse) résidants à **Tunis** qui ont participé à un voyage organisé au même jour que le voyage C0103 ?

Bon Travail



Département des Technologies de l'Informatique

REPUBLIQUE TUNISIENNE
Ministère de l'Enseignement
Supérieur,
de la Recherche Scientifique et de la
technologie
Direction Générale des Etudes
Technologiques

Classes DSI 21-22

Examen

Semestre 1 (A.U.2019-2020)

Le xx/01/2020 SI

Programmation
Orienté objet

Enseignants : M. Boukchim – A. Mechmech

Documents Non Autorisé

Questions de Cours

Question 1 :

```
public class Parent {
    int x;
    Parent(int k) {x=k;}
    int ajoute(int a) {return x+a;}
    public void moi() {System.out.println(" x = "+x);}
}

public class Enfant1 extends Parent {
    int y;
    Enfant1 (int k, int l) {super(k); y=l;}
    int ajoute(int a) {return x+2*a;}
}

public class Enfant2 extends Enfant1 {
    int z;
    Enfant2 (int k, int l, int m) {super(k, l); z= m;}
    int ajoute(int a) {return x+3*a;}
    public void moi() {super.moi();System.out.println(" z = "+z);}
}

public class Essai{
    public static void main (String args[]) {
        int a =2;
        Parent p = new Parent(3);
        p.moi();
        System.out.println(" ajoute("+ a +") = "+ p.ajoute(a) );
        Enfant1 e1 = new Enfant1(3, 4);
        e1.moi();
        System.out.println(" ajoute("+ a +") = "+ e1.ajoute(a) );
        e1 = new Enfant2(3, 4, 5);
        e1.moi();
        System.out.println(" ajoute("+ a +") = "+ e1.ajoute(a) );}
}
```

- a) Quels sont les attributs dont disposent les classes Enfant1 et Enfant2?
- b) Ecrivez le résultat de l'exécution de la classe Essai.

Question 2 :

Proposez une correction du code suivant :

```
public class A {  
    protected int k;  
    private int j;  
    void afficher() { System.out.println(+ j); }  
    public int get() { return i; }  
    public int get() { return i; }  
    public void set(int i) { this.i=i; }  
    public void set(int j) { this.j=j; }  
}  
  
public class B extends A {  
    public int i;  
    void afficher() { System.out.println(+ j+k); }  
}
```

Question 3 :

Soit le code Java suivant:

```
public PolygoneSegment(Point[] t){  
    segments = new Segment[t.length];  
    for (int i=0; i<segments.length; i++){  
        segments[i] = new Segment(t[i], t[(i+1) % t.length]);  
    }  
}
```

Indiquez comment doit être déclaré l'objet « segments ».

Exercice :

NB : Tous les champs doivent être déclarés **private** toutes les méthodes doivent être déclarés **public**.

Soit une Banque qui veut mettre en place un système informatique pour la gestion des Comptes de ces Clients.

Question 1 : Gestion des Clients

Un Client est caractérisé par son **Nom** (chaîne de caractère) et sa **date de naissance** (trois entiers pour le jour, le mois et l'année).

Ecrire une classe Client qui permet de définir :

- les attributs d'un Client
- Un constructeur associé
- Une méthode d'affichage **public void Afficher()** qui affiche les champs d'une personne
- Et enfin, une méthode **getnom()**.

Question 2 : Gestion des Comptes

On se propose de décrire simplement les informations nécessaires à la création d'un compte bancaire.

On distinguera deux types de comptes, les comptes courants et les comptes épargnes qui seront implémentés par deux classes distinctes **CompteCourant** et **CompteEpargne** dérivant toutes les deux d'une même classe abstraite **Compte**.

Les comptes épargnes ont un montant minimal qui sera représenté par un champ **depotMinimal**.

La classe abstraite **Compte**, comporte les champs suivants :

- Une **personne p** de type **personne**
- Un entier **numéro** représentant le **numéro du compte**.
- Un **réel montant** représentant le **montant du compte**.

Ecrire la classe abstraite **Compte** qui permet de définir :

- Un constructeur.
- Une méthode abstraite **retrait(float som)** : Simulant le retrait d'un compte.
- Une méthode **depot(float som)** : Simulant le dépôt d'une somme sur un compte.
- Une méthode **getpersonne()**.
- Une méthode **affiche()** qui affiche les champs de la personne et du compte.

Question 3 : Gestion Comptes Epargne

Rappelons que les comptes épargnes ont un montant minimal qui sera représenté par un champ **depotMinimal**.

Sur un compte épargne, le retrait ne sera effectué que si l'opération n'amène pas en dessous de la valeur minimale (**depotMinimal**).

Ecrire la classe **CompteEpargne** qui dérive de **Compte** qui permet de :

- Développer le constructeur.
- Redéfinir la méthode **affiche()**

Question 4 : Gestion de la Banque

Ecrire la classe **Banque** qui contient :

- Un tableau de comptes
- Un entier qui représente le nombre de comptes existants dans le tableau.
- Un constructeur qui permet de créer un tableau de 100 éléments max, et initialiser le nombre de comptes à 0.
- Une méthode **void AjoutCompte (Compte C)** qui permet d'ajouter un compte en fin du tableau.
- Une méthode **void afficheComptesClient(Personne p)** qui permet d'afficher tous les comptes d'une personne donnée.